LevelUp Cloud Trainings
Sateesh Pabbathi | Microsoft Certified Trainer (MCT)

PySpark DataFrame

Module 05 — File Formats: Read & Write

Expert Class Notes

CSV | JSON | Parquet | Text | ORC | Read Options | Schema Definition | Bad Records | Write Modes | partitionBy | Performance | MCQ Interview Questions

5 File Formats	Read Options	Write & Partition	MCQ Bank
CSV, JSON, Parquet, Text, ORC	header, schema, bad records	modes, partitionBy, performance	15 exam questions + answers

www.levelupedu.net

ABOUT THE AUTHOR

Microsoft Certified Trainer | Data Engineering Educator | 22+ Years Industry Experience

Sateesh Pabbathi
MCT
www.levelupedu.net

Sateesh Pabbathi

Microsoft Certified Trainer (MCT)

With 22+ years in data and cloud, Sateesh is the founder of LevelUp Cloud Trainings — India's premier data engineering platform with 5,000+ trained students.

DataFrame 05 covers the complete file format ecosystem in PySpark — the foundation of every data engineering pipeline. Every Flipkart ETL, every HDFC Bank data lake, every TCS data warehouse reads and writes using exactly these techniques.

Core Expertise

- Microsoft Fabric (DP-600) | Apache Spark & PySpark
- Databricks | Azure Data Factory | GCP & AWS
- SQL Server | Python | Big Data & Hadoop Ecosystem

"I don't just teach technology — I build careers."

5,000+	22+	10+
Students Trained	Years Experience	Technologies Taught

Module 1: Why File Formats Matter — The Foundation

Every data engineering pipeline starts and ends with files. Choosing the WRONG file format for production is one of the most common and costly mistakes junior engineers make. This module gives you the full picture: what each format is, when to use it, and why Parquet dominates production systems.

> **Sateesh Says 💡**
>
> *Think of file formats like modes of transport for data. CSV is an auto-rickshaw: simple, everyone understands it, but slow and no structure.*
>
> *JSON is a courier box: flexible shape, can hold complex objects, but heavy and verbose.*
>
> *Parquet is a bullet train: fast, compressed, columnar — the choice for TBs of data at Flipkart, Zomato, and every production data lake.*
>
> *ORC is the rival bullet train: also fast and columnar, preferred in Hive/Hadoop. Databricks defaults to Delta Lake (built on Parquet).*
>
> *Choose your transport based on the data size, query pattern, and who needs to read it!*

1.1 The 5 File Formats — Quick Overview

Format	Type	Schema Stored?	Compression	Best For
CSV	Row-based text	No	None (large)	Human-readable, legacy systems, small data export
JSON	Row-based text	No (inferred)	None (very large)	Semi-structured data, APIs, nested objects
Parquet	Columnar binary	YES — embedded	Snappy/Gzip (4-10x)	Production analytics, data lakes, Spark default
ORC	Columnar binary	YES — embedded	Zlib/Snappy (4-10x)	Hive-heavy pipelines, Hadoop ecosystem
Text	Plain text lines	No	None	Log files, simple line-by-line processing

1.2 The Universal Read/Write Syntax — Two Styles

PySpark gives you two styles to read files. Both are equivalent — know both for interviews and production. The class uses both styles.

Style 1: format().option().load() — verbose but explicit (from class)

```
# Style 1: Generic format API — works for ALL formats
```

```
df = spark.read \
     .format('csv') \                # specify format
     .option('header', True) \       # add options one by one
     .option('inferSchema', True) \
     .load('/content/sample_data.csv')  # path last
df.show()

# With inferSchema — reads schema types automatically
df = spark.read \
     .format('csv') \
     .option('header', True) \
     .option('inferSchema', True) \
     .load('/content/sample_data.csv')
df.show()
df.printSchema()
```

Style 2: Shorthand direct methods — concise (also from class)

```
# Style 2: Direct shorthand — format-specific method
df = spark.read.csv('/content/sample_data.csv', header=True)
df = spark.read.json('/content/sample_data.json')
df = spark.read.parquet('/content/sample_data.parquet')
df = spark.read.text('/content/file.txt')
df = spark.read.orc('/content/file.orc')

# BOTH styles produce IDENTICAL DataFrames and execution plans!
# MCQ TRAP: Q from class — 'Which method reads CSV with headers?'
# Answer: ALL of these are correct (from class MCQ d = 'all above')
# a) spark.read.csv('file.csv', header=True)
# b) spark.read.format('csv').option('header','true').load('file.csv')
# c) spark.read.option('header','true').csv('file.csv')
```

Module 2: CSV — Comma Separated Values

TXT
CSV

Row-based plain text. Universal but slow, no schema, no compression.

Sateesh Says 💡

CSV is like a typed letter — everyone can read it, but it's slow to process and has no intelligence built in.

At every company, CSV is the ENTRY POINT for data from legacy systems, Excel, and external partners.

Your job as a data engineer: ingest CSV, validate it, convert it to Parquet. Never store production data as CSV!

2.1 Reading CSV — All Options Explained

Complete CSV reading with all options (from class + extended)

```
# From class: basic read with format API
df = spark.read.format('csv') \
     .option('header', True)  \
     .load('/content/sample_data.csv')
df.show()

# From class: with inferSchema
df = spark.read.format('csv') \
     .option('header', True) \
     .option('inferSchema', True) \
     .load('/content/sample_data.csv')
df.show()
df.printSchema()  # now shows int/double/date types, not all string

# ALL CSV options — production setup:
df = spark.read \
    .option('header',      'true') \    # 1st row = column names
    .option('inferSchema', 'true') \    # auto-detect types
    .option('sep',         ',') \       # delimiter (default comma)
    .option('quote',       '"') \      # quote character
    .option('escape',      '\\') \    # escape character
    .option('nullValue',   'NA') \      # treat 'NA' as null
    .option('emptyValue',  '') \        # empty string handling
    .option('dateFormat',  'dd/MM/yyyy') \ # parse date columns
    .option('multiline',   'true') \    # handle multi-line fields
    .option('encoding',    'UTF-8') \   # file encoding
    .option('mode',        'PERMISSIVE') \ # bad record mode
    .csv('/content/sample_data.csv')
```

2.2 CSV with Custom Delimiter

Reading semicolon-delimited or pipe-delimited files

```
# Semicolon-delimited (common in European CSVs / SAP exports)
df = spark.read \
    .option('delimiter', ';') \
    .csv('/content/european_data.csv', header=True, inferSchema=True)

# Pipe-delimited (common in banking / insurance)
df = spark.read \
    .option('sep', '|') \
    .csv('/content/bank_transactions.csv', header=True)

# Tab-delimited (TSV files)
df = spark.read \
    .option('sep', '\t') \
    .csv('/content/data.tsv', header=True, inferSchema=True)
```

2.3 Writing CSV — From Class

Writing DataFrame to CSV

```
# From class: write to CSV
df.write.csv('/content/sample_data1.csv')

# With options:
df.write \
  .option('header', 'true') \     # include header row
  .option('delimiter', ',') \     # specify delimiter
  .option('quoteAll', 'true') \   # quote all string values
  .mode('overwrite') \            # overwrite if exists
  .csv('/content/output.csv')

# Reduce to single file before writing (for small outputs)
df.coalesce(1) \
  .write.option('header','true').csv('/content/single_output.csv')
```

Common Mistake — Watch Out!

CSV write produces multiple part files: part-00000.csv, part-00001.csv, etc. — one per partition.

Reading CSV does NOT read these part files properly if you specify the directory path — use spark.read.csv('dir/') to read ALL parts.

inferSchema=True reads the ENTIRE CSV twice — once to detect types, once to load. For large files always provide explicit schema.

CSV has no built-in compression, no schema, no column pruning. Never use CSV as your production storage format.

Module 3: JSON — JavaScript Object Notation

{ }
JSON

Semi-structured text format. Supports nested objects and arrays. No schema stored.

Sateesh Says 💡

JSON is the language of APIs. Every Zomato order, every Paytm transaction, every mobile app event arrives as JSON.

PySpark automatically infers nested JSON into StructType columns — it handles struct, array, and nested arrays out of the box!

The big gotcha: standard JSON files have all records in ONE array (multiline). JSON Lines format has one record per line. Know which you have!

3.1 Reading JSON — Two Formats

Reading JSON — from class and all patterns

```
# From class: read JSON using format API
df = spark.read.format('json').load('/content/sample_data.json')
df.show()

# From class: shorthand
df = spark.read.json('/content/sample_data.json')
df.show()

# JSON LINES format (one record per line — most common with APIs)
# {"id":1, "name":"Raj", "city":"Mumbai"}
# {"id":2, "name":"Priya", "city":"Delhi"}
df = spark.read.json('/content/events.jsonl')   # auto-detected

# Standard JSON format (all records in one array) — needs multiline
# [
#   {"id":1, "name":"Raj"},
#   {"id":2, "name":"Priya"}
# ]
df = spark.read \
    .option('multiline', 'true') \   # REQUIRED for array JSON
    .json('/content/standard.json')

# Nested JSON — Spark auto-creates struct columns
# {"id":1, "address":{"city":"Mumbai","pin":400001}, "orders":[101,102]}
df = spark.read.json('/content/users.json')
df.printSchema()
# |-- id: long
# |-- address: struct
# |    |-- city: string
# |    |-- pin: long
# |-- orders: array
# |    |-- element: long
```

```
# Access nested fields
df.select('address.city', 'address.pin').show()
df.select(F.col('orders')[0].alias('first_order')).show()
```

3.2 Writing JSON

Writing DataFrame to JSON

```
# Basic write
df.write.json('/content/sample_data_out.json')
# Creates: part-00000.json, part-00001.json (one per partition)

# With mode and compression:
df.write \
  .mode('overwrite') \
  .option('compression', 'gzip') \
  .json('/content/output_compressed.json')
# Creates: part-00000.json.gz
```

Module 4: Parquet — The Production Standard

PARQUET

Columnar binary format. Self-describing schema. Built-in compression. Spark's native format.

Sateesh Says 💡

If PySpark is the engine of a Formula 1 car, Parquet is the fuel. Together they are unbeatable.

Parquet is the DEFAULT format in Databricks, Azure Data Lake, AWS S3-based data lakes, and GCP BigQuery storage.

When Flipkart processes 10 TB of orders daily — it's all stored as Parquet. When HDFC analyzes 500 million transactions — Parquet.

The class practical exercise: read CSV, filter, write Parquet — that is the EXACT pattern used in 90% of real ETL pipelines worldwide!

4.1 Why Parquet? — Technical Advantages

Feature	CSV	Parquet	Impact
Storage	~1x size	~5-10x smaller	1 TB CSV becomes ~200 GB Parquet
Schema	Not stored	Embedded in file	No inferSchema needed, no type errors
Column reads	ALL columns always	Only queried cols	SELECT 2 of 100 cols = 98% less I/O
Row filtering	Scan all rows	Row group stats used	Skip entire row groups by min/max
Compression	None	Snappy/Gzip built-in	Fast reads + small files
Partitioning	Manual folders	Native support	year=2024/month=01 partition pruning
Write speed	Fast	Slightly slower	Trade-off: slower write, much faster read
Read speed	Slow (parse text)	Very fast (binary)	10-100x faster for analytics

4.2 Reading Parquet — From Class

Reading Parquet files — from class

```
# From class: read full parquet directory (production way)
df = spark.read.format('parquet').load('/content/sample_data.parquet')
df.show()
```

```
# From class: read specific part file (class debugging pattern)
df33 = spark.read.parquet(
    '/content/person_data.parquet/' +
    'part-00000-317c95fe-5c78-4429-bbbf-794e8236335f-c000.snappy.parquet'
)
df33.collect()

# CORRECT production way — read the DIRECTORY (reads all parts):
df = spark.read.parquet('/content/person_data.parquet/')
df.show()

# With schema (optional — Parquet embeds its own schema):
df = spark.read.schema(my_schema).parquet('/path/to/data.parquet')

# Read partitioned Parquet (partition pruning happens automatically!)
df = spark.read.parquet('/datalake/orders/')
# If partitioned by year: /datalake/orders/year=2024/month=01/...
df.filter(col('year') == 2024).show()  # reads ONLY year=2024 folder
# Partition pruning: skips all other year folders = massive speedup!
```

4.3 Writing Parquet — Class Practical Exercise

The class practical: read CSV, filter rows, write as Parquet. This is the canonical ETL pipeline pattern.

Class exercise: CSV -> Filter -> Parquet

```
# FROM CLASS PRACTICAL EXERCISE:
# Step 1: Read CSV
df = spark.read.csv('/content/person_data.csv', header=True)

# Step 2: Filter (age < 30)
filter_df = df.filter(df['age'] < 30)

# Step 3: Write as Parquet
filter_df.write.parquet('/content/filtered1.parquet')
# Creates: /content/filtered1.parquet/part-00000-...snappy.parquet

# Step 4: Read back to verify (from class)
filter_output1 = spark.read.parquet('/content/filtered1.parquet/')
filter_output1.show()

# Also from class: write all person data as parquet
df.write.parquet('/content/person_data.parquet')

# Production write with all options:
df.write \
  .mode('overwrite') \                    # overwrite if exists
  .option('compression', 'snappy') \      # snappy = balanced speed
  .parquet('/datalake/orders/')

# Or using format API:
df.write \
  .format('parquet') \
  .mode('overwrite') \
```

```
    .save('/datalake/orders/')
```

Parquet file naming convention (from class)

```
After write, directory contains:
/content/filtered1.parquet/
  _SUCCESS                                      <- job completed OK
  part-00000-28ca61f7-ea3e-4725-...snappy.parquet  <- partition 0 data
  part-00001-28ca61f7-ea3e-4725-...snappy.parquet  <- partition 1 data

File naming: part-{partition}-{UUID}-c000.snappy.parquet
.snappy = Snappy compression (default for Parquet)
```

Module 5: Text Files and ORC Format

5.1 Reading Text Files

ABC
TEXT

Each line = one record. Single column 'value'. For log files and raw text.

Text file reading and writing

```
# Read text file — creates ONE column named 'value' (string)
df = spark.read.text('/content/file.txt')
df.show()
df.printSchema()
# root
#  |-- value: string (nullable = true)  <- single column, all lines

# Each line of the file becomes one row in 'value' column
# Example file content:
# Hello World
# How are you
# This is PySpark
# df.show() output:
# +------------------+
# |value             |
# +------------------+
# |Hello World       |
# |How are you       |
# |This is PySpark   |

# Process the text — split into fields, parse JSON lines, etc.
from pyspark.sql.functions import split, col
df.select(split(col('value'), ',')[0].alias('field1'),
          split(col('value'), ',')[1].alias('field2')).show()

# Write text (column must be a single StringType column)
df.write.text('/content/output_text/')
```

5.2 ORC — Optimized Row Columnar Format

ORC
ORC

Columnar binary. Hive ecosystem favourite. Strong compression and predicate pushdown.

ORC reading and writing

```
# Reading ORC files
df = spark.read.orc('/content/file.orc')
df.show()

# ORC with schema inference (it's self-describing like Parquet)
```

```
df = spark.read.format('orc').load('/path/to/file.orc')
df.printSchema()  # schema embedded in file — no inferSchema needed

# Writing ORC
df.write.orc('/content/output.orc')
df.write.mode('overwrite').orc('/content/output.orc')
df.write.format('orc').mode('overwrite').save('/content/output.orc')
```

Parquet	ORC
Open-source — Apache Arrow ecosystem	Open-source — Apache Hive ecosystem
Default in Spark, Databricks, Delta Lake	Default in Hive, older Hadoop pipelines
Better for Spark workloads (native)	Better for Hive SQL workloads
Snappy compression (default)	Zlib compression (default, higher ratio)
Column pruning + predicate pushdown	Column pruning + predicate pushdown
Nested types: struct, array, map	Nested types: struct, list, map
Production choice for cloud data lakes	Use if already in HiveQL-heavy ecosystem

Module 6: Read Options — Full Reference

Read options are the knobs that control HOW Spark reads your files. Knowing all the important options and when to use them is essential for handling real-world messy data.

6.1 Schema Definition — Explicit vs Inferred

Sateesh Says 💡

inferSchema=True is your enemy in production. Here's why: it reads the ENTIRE file twice to guess types.

For a 100 GB CSV file: inferSchema makes Spark read 200 GB total. That is 2x the cost of just reading!

Always define an explicit schema in production. It runs ONCE, is deterministic, and prevents wrong type inference.

Real bug: a column of employee IDs like '001','002','003' gets inferred as Integer — losing the leading zeros. Explicit schema prevents this.

Explicit schema definition — from class

```
from pyspark.sql.types import StructType, StructField, StringType
from pyspark.sql.types import IntegerType, DoubleType, DateType
from pyspark.sql.types import TimestampType, BooleanType, LongType

# FROM CLASS: define schema before reading
schema = StructType([
    StructField('name',   StringType(),  True),  # True = nullable
    StructField('age',    IntegerType(), True),
    StructField('salary', IntegerType(), True)
])

df = spark.read.schema(schema).csv('/content/file.csv', header=True)
df.show()

# Full production schema for employee data
employee_schema = StructType([
    StructField('emp_id',     StringType(),    False), # NOT nullable
    StructField('name',       StringType(),    True),
    StructField('department', StringType(),    True),
    StructField('salary',     DoubleType(),    True),
    StructField('join_date',  DateType(),      True),
    StructField('is_active',  BooleanType(),   True),
    StructField('last_login', TimestampType(), True),
])

# Read CSV with explicit schema — FASTEST, most reliable
df = spark.read \
    .schema(employee_schema) \
    .option('header', 'true') \
    .option('dateFormat', 'dd/MM/yyyy') \
    .csv('/content/employees.csv')
```

6.2 Bad Records Handling — Three Modes

In production, data is messy. CSV files from SAP, Oracle exports, and flat files always have some corrupt or malformed records. The 'mode' option tells Spark what to do with them.

PERMISSIVE (Default)	DROPMALFORMED	FAILFAST
Corrupt rows are included in output Bad fields set to null Other valid rows still loaded Best for: large data where some bad records are acceptable Never crashes — safest option	Corrupt rows SILENTLY DROPPED Clean rows fully loaded No error thrown Best for: when bad records are expected and ignorable Good default for production pipelines	THROWS exception on first bad record Job fails immediately No partial output written Best for: when any bad record = critical failure Use for strict data contracts / financial data

Bad records mode — from class

```
# PERMISSIVE (default) — null for bad fields
df = spark.read \
    .option('mode', 'PERMISSIVE') \
    .csv('/content/file.csv', header=True)

# DROPMALFORMED — silently remove bad rows
df = spark.read \
    .option('mode', 'DROPMALFORMED') \
    .csv('/content/file.csv', header=True)

# FAILFAST — crash on first bad record
df = spark.read \
    .option('mode', 'FAILFAST') \
    .csv('/content/file.csv', header=True)

# PERMISSIVE with bad record column — capture corrupt rows
from pyspark.sql.types import *
schema_with_corrupt = StructType([
    StructField('name',    StringType(), True),
    StructField('age',     IntegerType(), True),
    StructField('_corrupt_record', StringType(), True)  # captures bad
])
df = spark.read \
    .schema(schema_with_corrupt) \
    .option('mode', 'PERMISSIVE') \
    .option('columnNameOfCorruptRecord', '_corrupt_record') \
    .csv('/content/file.csv', header=True)
# Bad rows will have their raw string in _corrupt_record column
df.filter(col('_corrupt_record').isNotNull()).show()  # see bad rows
```

6.3 All Read Options — Complete Reference Table

Option	Applies To	Values	What It Does
header	CSV	true / false	First row is column names (default: false)

inferSchema	CSV/JSON	true / false	Auto-detect column types (slow — 2x scan)
sep / delimiter	CSV	',' '|' '\t' ';'	Column separator character (default: ,)
quote	CSV	'"' '"''	Character wrapping quoted fields
multiline	JSON	true / false	JSON spans multiple lines (array format)
mode	CSV/JSON	PERMISSIVE DROPMALFORMED FAILFAST	How to handle corrupt/malformed records
nullValue	CSV	'NA' 'NULL' ''	String treated as null when reading
dateFormat	CSV/JSON	'dd/MM/yyyy'	Format to parse date columns
timestampFormat	CSV/JSON	'dd/MM/yyyy HH:mm:ss'	Format to parse timestamp columns
encoding	CSV	'UTF-8' 'ISO-8859-1'	File character encoding
columnNameOfCorruptRecord	CSV/JSON	'_corrupt'	Column to capture bad records
compression	All write	'snappy' 'gzip' 'none'	Output file compression

Module 7: Writing DataFrames — Modes, Formats & partitionBy

Writing is the final step of every ETL pipeline. The write mode determines what happens when output already exists. The partitionBy option is one of the most powerful performance tools in the entire Spark ecosystem.

> **Sateesh Says 💡**
>
> *In production data engineering, you run the same ETL job daily. The write MODE decides: do you replace yesterday's data? Add to it? Skip if already there?*
>
> *Getting the write mode wrong = double-counting data (append instead of overwrite) or losing data (overwrite instead of append).*
>
> *partitionBy is the secret weapon of every senior data engineer — it turns a 1-hour query into a 2-minute query through partition pruning!*

7.1 Write Modes — The Four Options

Mode	Behavior	Error If Exists?	Use Case
overwrite	Delete existing data, write new data	No — replaces	Daily full refresh of reference tables
append	Add new data to existing output	No — adds more	Incremental loads, event streaming
ignore	Write ONLY if path doesn't exist	No — skips silently	Idempotent pipelines, checkpoint writes
error (default)	Fail immediately if output exists	YES — throws error	Development safety net, prevent accidental overwrite

Write modes in practice

```
# overwrite — most common in production ETL
df.write.mode('overwrite').parquet('/datalake/daily_orders/')

# append — for incremental loads and streaming
df.write.mode('append').parquet('/datalake/all_events/')

# ignore — idempotent pipeline (run once, skip if already done)
df.write.mode('ignore').parquet('/datalake/processed/2024-01-01/')

# error (default) — safely fails if you re-run accidentally
df.write.parquet('/datalake/critical_report/')  # mode='error'
# Use in dev to prevent accidental overwrites

# Combining mode + format:
df.write.mode('overwrite').format('parquet').save('/output/')
```

```
df.write.mode('overwrite').parquet('/output/')   # same thing
```

7.2 partitionBy — The Performance Game Changer

partitionBy splits the output into directory folders based on column values. When reading back, Spark can SKIP entire folders based on query conditions — called partition pruning. This turns full table scans into targeted folder reads.

partitionBy — from class and production patterns

```
# FROM CLASS: partitionBy with overwrite mode
df.write.mode('overwrite').partitionBy('year') \
  .parquet('/path/to/table')

# What it creates on disk:
# /path/to/table/
#   year=2022/
#     part-00000.snappy.parquet
#   year=2023/
#     part-00000.snappy.parquet
#   year=2024/
#     part-00000.snappy.parquet

# Multi-level partitioning (very common in production)
df.write.mode('overwrite') \
  .partitionBy('year', 'month', 'day') \
  .parquet('/datalake/orders/')
# Creates: /datalake/orders/year=2024/month=01/day=15/part-*.parquet

# Partition by business keys
df.write.mode('overwrite') \
  .partitionBy('country', 'department') \
  .parquet('/datalake/employees/')
# Creates: /employees/country=India/department=IT/part-*.parquet
```

7.3 How Partition Pruning Works — The Magic

Partition pruning — reading only what you need

```
# Data stored as: /datalake/orders/year=2024/month=01/day=15/...

# Read ALL data (no pruning) — scans every folder
df = spark.read.parquet('/datalake/orders/')
df.count()   # reads ALL years, months, days — slow!

# With partition filter — Spark ONLY reads year=2024 folder
df = spark.read.parquet('/datalake/orders/')
df.filter(col('year') == 2024).count()   # reads 1/3 of data!

# Multi-level pruning — reads ONLY Jan 15, 2024 folder
df.filter((col('year')==2024) &
          (col('month')==1)   &
          (col('day')==15)).show()
```

```
# Spark reads: /datalake/orders/year=2024/month=1/day=15/ ONLY!
# All other 364 days are NEVER read — massive performance gain!

# Verify pruning with explain():
df.filter(col('year')==2024).explain()
# Look for: PartitionFilters: [isnotnull(year#0), (year#0 = 2024)]
```

Pro Tip

Choose partition columns carefully: year/month/day is ideal (low cardinality, high query filter frequency).

Never partition by high-cardinality columns like order_id or customer_id — you'll create millions of tiny files (the small file problem).

Rule of thumb: partition column should have 10-10,000 unique values. More = small file problem. Less = no pruning benefit.

In Databricks: use OPTIMIZE and ZORDER for fine-grained data skipping within partitions.

7.4 Handling Large Files — Repartition Before Write

Repartition for performance — from class agenda

```
# FROM CLASS: specifying partitions while reading
df = spark.read.csv('/content/large_file.csv',
                    header=True, inferSchema=True).repartition(5)
# .repartition(5) splits into 5 partitions for parallel processing

# Control output file count before writing
# Too many small files (common with streaming):
df.coalesce(10).write.mode('overwrite').parquet('/output/')
# coalesce(10) = max 10 output files, no shuffle

# Repartition for even distribution before write:
df.repartition(20).write.mode('overwrite').parquet('/output/')
# repartition(20) = exactly 20 files, full shuffle, even sizes

# Repartition by partition column BEFORE partitionBy write
df.repartition(col('year')) \
  .write.mode('overwrite').partitionBy('year').parquet('/output/')
# Groups same-year data on same Executors before write = efficient
```

Module 8: Complete Read & Write Reference — All 5 Formats

8.1 Reading All Formats — Side by Side

All 5 read methods in one reference block

```
# CSV
df = spark.read.csv('path/file.csv', header=True, inferSchema=True)
df = spark.read.format('csv').option('header','true').load('file.csv')

# JSON
df = spark.read.json('path/file.json')
df = spark.read.format('json').load('path/file.json')
df = spark.read.option('multiline','true').json('array_format.json')

# Parquet
df = spark.read.parquet('path/data.parquet')
df = spark.read.format('parquet').load('path/data.parquet')

# Text
df = spark.read.text('path/file.txt')
df = spark.read.format('text').load('path/file.txt')

# ORC
df = spark.read.orc('path/file.orc')
df = spark.read.format('orc').load('path/file.orc')
```

8.2 Writing All Formats — Side by Side

All 5 write methods in one reference block

```
# CSV (from class)
df.write.csv('/content/sample_data1.csv')
df.write.mode('overwrite').option('header','true').csv('/output/')

# JSON
df.write.json('/output/data.json')
df.write.mode('overwrite').json('/output/data.json')

# Parquet (from class practical)
df.write.parquet('/content/person_data.parquet')
filter_df.write.parquet('/content/filtered1.parquet')
df.write.mode('overwrite').parquet('/output/')

# Text
df.write.text('/output/text/')
# NOTE: DataFrame must have single StringType column for text write

# ORC
```

```
df.write.orc('/output/data.orc')
df.write.mode('overwrite').orc('/output/data.orc')

# partitionBy — works with ALL columnar formats (from class MCQ)
df.write.mode('overwrite').partitionBy('year').parquet('/path/')
```

Module 9: MCQ Interview Questions — From Class

The class notebook contains multiple MCQ questions that are directly from real PySpark and Databricks certification exams. Every answer is explained in detail so you understand WHY, not just WHAT.

Sateesh Says 💡

MCQs from this class appear verbatim in Databricks Certified Associate Developer exam and TCS/Infosys technical rounds.

Read the explanation carefully — interviewers always ask follow-up 'why' questions after the MCQ!

Sateesh said in class: 'Know the REASON behind each answer. That's what separates a ₹6 LPA offer from a ₹18 LPA offer.'

Q1 — Which PySpark method is used to read a CSV file with headers into a DataFrame?

a) spark.read.csv('file.csv', header=True)
b) spark.read.format('csv').option('header','true').load('file.csv')
c) spark.read.option('header','true').csv('file.csv')
d) All of the above

Answer: d) All of the above

All three are valid and produce identical DataFrames. Style 1 is shorthand, Style 2 is the verbose format API, Style 3 mixes both. In production you'll see all three — know each one.

Q2 — What is the correct way to register a DataFrame as a temporary view in PySpark?

a) df.createTempView('my_view')
b) df.createOrReplaceTempView('my_view')
c) df.registerTempTable('my_view')
d) Both a and b are correct

Answer: d) Both a and b are correct

createTempView() creates a new view but FAILS if the view already exists.
createOrReplaceTempView() creates or replaces atomically — safer for repeated runs.
registerTempTable() is deprecated (Spark 1.x). In production, always use createOrReplaceTempView().

Q3 — Which Spark SQL function would you use to parse a JSON string column?

a) json_parse()
b) from_json()
c) parse_json()
d) json_extract()

Answer: b) from_json()

from_json(col, schema) converts a JSON string column into a StructType column. Example: df.withColumn('data', from_json(col('json_str'), schema)). to_json() does the reverse — StructType to JSON string. json_parse(), parse_json(), json_extract() do not exist in PySpark.

Q4 — In PySpark, how do you select specific columns from a DataFrame?

a) df.select('col1', 'col2')
b) df.select(df.col1, df.col2)
c) df.select(F.col('col1'), F.col('col2'))
d) All of the above

Answer: d) All of the above

All three styles work and produce identical DataFrames. String names are simplest. df.colname style is DataFrame-bound (can cause issues in joins). F.col() is most flexible for complex expressions. Mix them freely in the same select() call.

Q5 — What does df.write.mode('overwrite').partitionBy('year').parquet('/path/to/table') do?

a) Writes data in append mode with year partitioning
b) Writes data in overwrite mode with year partitioning
c) Creates a new table partitioned by year
d) Updates existing partitions for the specified year

Answer: b) Writes data in overwrite mode with year partitioning

mode('overwrite') deletes existing output and writes fresh data. partitionBy('year') creates year=XXXX/ subdirectories. This REPLACES all existing data in the path. Note: in Databricks Delta Lake, mode('overwrite') + partitionBy replaces only matching partitions by default — different from plain Parquet overwrite.

Q6 — Which SQL command creates a managed table in Databricks?

a) CREATE TABLE table_name USING DELTA LOCATION '/path'
b) CREATE TABLE table_name (col1 STRING, col2 INT) USING DELTA
c) CREATE OR REPLACE TABLE table_name AS SELECT * FROM source
d) Both b and c are correct

Answer: d) Both b and c are correct

Option a creates an EXTERNAL table (LOCATION keyword = external). Managed tables have their lifecycle managed by Spark — dropping the table drops the data. External tables only drop the metadata. In Databricks, the default format is Delta, so USING DELTA is often optional.

Q7 — Which PySpark transformation is used to remove duplicate rows?

a) df.distinct()
b) df.dropDuplicates()
c) df.drop_duplicates()
d) Both a and b are correct

Answer: d) Both a and b are correct

distinct() and dropDuplicates() with no arguments are identical. drop_duplicates() is a Pandas alias that also works in PySpark (Spark added it for Pandas compatibility). dropDuplicates(['col1','col2']) is more powerful — deduplicates based on subset of columns. distinct() always compares all columns.

Q8 — Which PySpark method is used to filter rows in a DataFrame?

a) df.filter(col('age') > 25)
b) df.where(col('age') > 25)
c) df.filter('age > 25')
d) All of the above

Answer: d) All of the above

filter() and where() are identical aliases. The condition can be: Column expression (col('age')>25), string SQL expression ('age > 25'), or DataFrame column reference (df['age']>25). All three produce identical execution plans. where() is favored by SQL-background developers.

Q9 — What does the following code do? df.withColumn('status', when(col('age') >= 18, 'Adult').otherwise('Minor'))

a) Filters rows where age >= 18
b) Creates a new column 'status' with conditional values
c) Updates the 'age' column with status values
d) Groups data by age ranges

Answer: b) Creates a new column 'status' with conditional values

withColumn() with a new column name ADDS a column (not replaces). when().otherwise() is PySpark's CASE WHEN — returns 'Adult' if age>=18, 'Minor' otherwise. Result: all original columns remain, PLUS a new 'status' column. If the column name already existed, it would be REPLACED.

Q10 — What does the collect() action do in PySpark?

a) Collects all rows from the DataFrame to the driver
b) Groups rows by a specified column
c) Counts the number of rows in the DataFrame
d) Removes duplicate rows from the DataFrame

Answer: a) Collects all rows from the DataFrame to the driver

collect() brings ALL rows as a Python list of Row objects to the Driver node. DANGER: on large DataFrames this causes OutOfMemoryError. Result is List[Row]. Access values: rows[0]['column_name'] or rows[0][0]. Use take(N) for safety. count() counts rows. groupBy() groups. distinct() deduplicates.

Q11 — In Spark SQL, which function is used to get the current date?

a) current_date()
b) now()
c) today()
d) getdate()

Answer: a) current_date()

current_date() returns today's date as a DateType column. current_timestamp() returns current date + time as TimestampType. now() exists in MySQL but not PySpark. today() and getdate() do not exist in PySpark. Usage: df.withColumn('today', current_date()).show()

Q12 — What is the correct way to add a new column with a constant value to a DataFrame?

a) df.withColumn('new_col', 'constant_value')
b) df.withColumn('new_col', lit('constant_value'))
c) df.select('*', lit('constant_value').alias('new_col'))
d) Both b and c are correct

Answer: d) Both b and c are correct

lit() wraps a Python literal into a Spark Column object. Option a is WRONG — passing a string directly would make Spark look for a column named 'constant_value'. Option b uses withColumn. Option c uses select with * plus the literal. Both b and c are correct. lit() is mandatory whenever you use a constant in a column expression.

Q13 — Which window function assigns row numbers to each row within a partition?

a) rank()
b) dense_rank()
c) row_number()
d) ntile()

Answer: c) row_number()

row_number() assigns UNIQUE sequential numbers (1,2,3,4) within a partition — no ties, always unique. rank() gives same number for ties but has gaps: (1,1,3,4). dense_rank() gives same number for ties with no gaps: (1,1,2,3). ntile(N) divides into N equal buckets. For 'keep latest record per ID', row_number() is the standard choice.

Q14 — What does df.groupBy('department').agg(count('*').alias('employee_count'), avg('salary').alias('avg_salary')) do?

a) Filters employees by department
b) Groups by department and calculates count and average salary
c) Sorts employees by department and salary
d) Creates a pivot table

Answer: b) Groups by department and calculates count and average salary

groupBy('department') groups rows by department. .agg() applies multiple aggregate functions simultaneously. count('*') counts all rows per group (including nulls). avg('salary') computes mean salary per group. Result: one row per department with employee_count and avg_salary columns. This is SQL GROUP BY equivalent.

Q15 — What is the primary storage format used in Databricks for structured data?

a) CSV
b) Parquet

c) JSON
d) ORC

Answer: b) Parquet (specifically Delta Lake, which is built on Parquet)

Databricks uses Delta Lake as its default storage format for managed tables — Delta is an open-source format BUILT ON TOP of Parquet with added ACID transactions, versioning, and time travel. Under the hood it is Parquet files + _delta_log/ JSON transaction log. For Spark outside Databricks: Parquet is still the standard choice.

Module 10: Master Reference & Practical Exercises

10.1 Complete File Format Decision Guide

When to use which format — decision tree

```
Is the data going to be read by humans directly?
-> YES: Use CSV or JSON
-> NO:  Use Parquet, ORC, or Delta

Is the data structured (tabular) or semi-structured (nested)?
-> Structured tables:  Parquet (Spark) or ORC (Hive)
-> Nested/flexible:    JSON or Delta with complex schema

Is this production in Databricks / Azure / AWS?
-> YES: Use Delta Lake (.write.format('delta')) or Parquet
-> Hive-heavy Hadoop: Use ORC

Do you need ACID transactions (upserts, deletes)?
-> YES: Use Delta Lake (not plain Parquet)
-> NO:  Plain Parquet is fine

Is this a log or line-by-line text processing task?
-> YES: Use Text format + flatMap + parse
-> NO:  Use structured format (Parquet/CSV)
```

10.2 The Complete ETL Pipeline — Putting It All Together

Full end-to-end pipeline: read CSV, transform, write Parquet with partitions

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, year, month
from pyspark.sql.types import *

spark = SparkSession.builder.appName('LevelUp_ETL').getOrCreate()

# STEP 1: Define schema explicitly (production best practice)
schema = StructType([
    StructField('emp_id',     StringType(),  False),
    StructField('name',       StringType(),  True),
    StructField('department', StringType(),  True),
    StructField('salary',     DoubleType(),  True),
    StructField('join_date',  DateType(),    True),
])

# STEP 2: Read CSV with explicit schema
df = spark.read \
    .schema(schema) \
    .option('header', 'true') \
```

```
        .option('mode', 'DROPMALFORMED') \  # drop bad records
        .csv('/input/employees.csv')

    # STEP 3: Add derived columns
    df2 = df \
        .withColumn('join_year',  year(col('join_date'))) \
        .withColumn('join_month', month(col('join_date')))

    # STEP 4: Filter and transform
    df3 = df2.filter(col('salary').isNotNull()) \
             .filter(col('department').isin(['IT','Finance','HR']))

    # STEP 5: Write partitioned Parquet to data lake
    df3.write \
       .mode('overwrite') \
       .partitionBy('join_year', 'department') \
       .parquet('/datalake/employees/')

    # Output structure:
    # /datalake/employees/join_year=2022/department=IT/part-*.parquet
    # /datalake/employees/join_year=2022/department=HR/part-*.parquet
    # /datalake/employees/join_year=2023/department=Finance/part-*.parquet

    # STEP 6: Verify by reading back with partition pruning
    result = spark.read.parquet('/datalake/employees/') \
                 .filter((col('join_year')==2023) &
                         (col('department')=='IT'))
    result.show()  # reads ONLY join_year=2023/department=IT folder!
```

Sateesh Says 💡

You have now mastered PySpark DataFrame 05 — every file format, every read/write option, all 5 write modes, partitionBy, and 15 MCQ questions.

Practice Task 1: Read the sample_data.csv using the format().option().load() style AND the shorthand style. Confirm they produce identical printSchema() output.

Practice Task 2: Read person_data.csv, filter age < 30, write as Parquet with mode=overwrite, read back and show. Match the class exercise exactly.

Practice Task 3: Write the same DataFrame to 4 formats (CSV, JSON, Parquet, ORC), compare file sizes. Observe how much smaller Parquet is.

Practice Task 4: Write a partitioned Parquet with year and month columns. Query with a year filter and use explain() to verify partition pruning is happening.

MCQ Self-Test: Cover the answers and try all 15 questions. Aim for 14/15 correct before your interview.

All the best! — Sateesh Pabbathi, MCT | www.levelupedu.net